

Matcher & Test Double

(with AssertJ, Mockito)

CONTENTS

목차

Chapter1

Matcher

Chapter2

Test Double (Mocking)

Chapter3

Google Mock 준비

Chapter4

Stubbing & Verify

Chapter5

Stubbing 응용

Chapter6

SPY 이해

Chapter7

Mock Injection

가독성있는 Unit Test를 위한 Library

AssertJ

AssertJ

JUnit 를 더 편하고 가독성 좋게 쓰기 위한 Library

- 가독성 높이고
- 더 유용한 오류 메시지 출력

```
@Test
public void getSum() {
    Cal c = new Cal();
    int ret = c.getSum(a: 10, b: 20);

    assertEquals(ret, actual: 30);
}
```

JUnit

```
@Test
public void getSum() {
    Cal c = new Cal();
    int ret = c.getSum(a: 10, b: 20);

    assertThat(ret).isEqualTo(30);
}
```

AssertJ

```
@Test
public void getSum() {
    Cal c = new Cal();
    int ret = c.getSum(a: 10, b: 20);

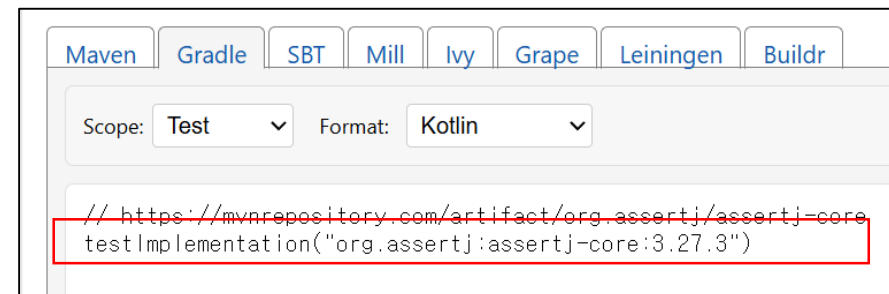
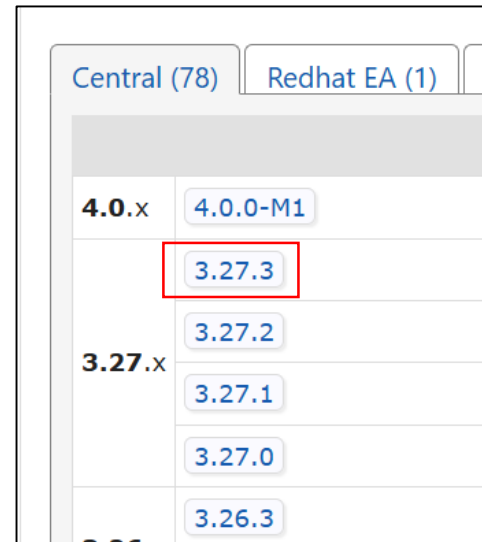
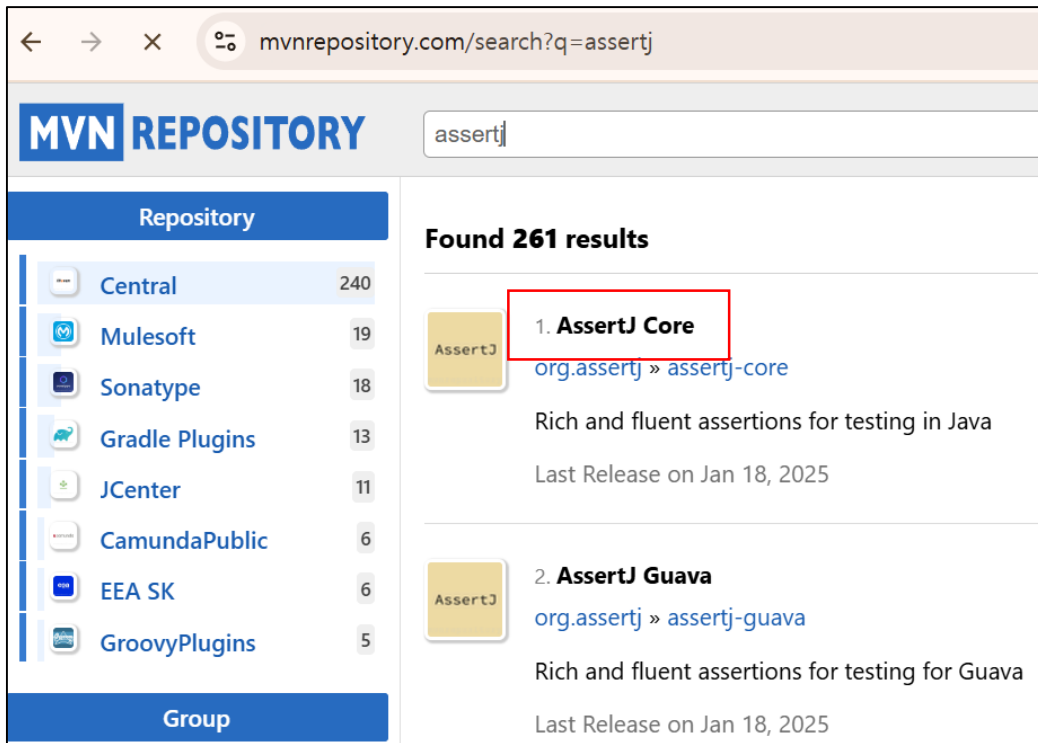
    assertThat(ret, equalTo(operand: 30));
}
```

Hamcrest

AssertJ 설치하기

MVN Repo 사이트 접속 후 assertj 검색

assertj 3.X 마지막 버전을 선택한다. (4.X 버전을 선택해도 상관없음)



build.gradle.kts에 복사하여 설치

AssertJ 테스트 1

AssertJ의 기본 Assertion 추가

IDE의 자동완성 기능을 이용하여 import / assertThat 을 완성한다.

```
1  import org.junit.jupiter.api.Test;
2
3  import static org.junit.jupiter.api.Assertions.*;
4
5  class GameTest {
6
7      @Test
8      void name() {
9
10         assertThat
11
12     }
13 }
```

자동완성 메뉴:

- Assertions.assertThat(...) (org.assertj.core.a
- AssertionsForClassTypes.assertThat
- AssertionsForInterfaceTypes.asser...
- ClassBasedNavigableIterableAssert...

assertThat 입력 후 Ctrl + Space 3회 입력

자동추가됨

```
import org.assertj.core.api.Assertions;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class GameTest {

    @Test
    void name() {

        Assertions.assertThat(1).isEqualTo(1);

    }
}
```

Test Results:

- ✓ Test Results
- ✓ GameTest
- ✓ name

AssertJ 테스트 2

Static Import 추가하기

기본 Assertion 대신 사용하기 위해 Static Import를 등록한다.

```
1 import org.assertj.core.api.Assertions;
2 import org.junit.jupiter.api.Test;
3
4 import static org.junit.jupiter.api.Assertions.*;
5
6 class GameTest {
7
8     @Test
9     void name() {
10
11         Assertions.assertThat(1).isEqualTo(1);
12
13     }
14 }
```

Extract Surround // ≡ ⋮

Introduce local variable

Add on-demand static import for 'org.assertj.core.api.Assertions' ⋮

Create new scratch file from selection

Alt + Enter 를 누른 후
static import 처리를 선택한다.

삭제

```
1 import org.assertj.core.api.Assertions;
2 import org.junit.jupiter.api.Test;
3
4 import static org.assertj.core.api.Assertions.*;
5 import static org.junit.jupiter.api.Assertions.*;
6
7 class GameTest {
8
9     @Test
10     void name() {
11         assertThat(1).isEqualTo(1);
12     }
13 }
```

삭제

✓ Test Results

✓ GameTest

✓ name

AssertJ 써보기

용어암기

Actual : 실제 대상
Expected : 기대값

메서드 체이닝으로 구현

1. `assertThat(실제대상).isEqualTo(기대값)`
2. `assertThat(실제대상).isNotEqualTo(아니길 바라는 값)`

```
@Test
public void getSum() {
    Cal c = new Cal();
    int ret = c.getSum( a: 10, b: 20);

    assertThat(ret).isEqualTo(30);
}
```

```
void getSum() {
    Cal c = new Cal();
    int ret = c.getSum( a: 10, b: 20);

    assertThat(ret).isNotEqualTo(29);
    assertThat(ret).isEqualTo(30);
    assertThat(ret).isNotEqualTo(31);
}
```

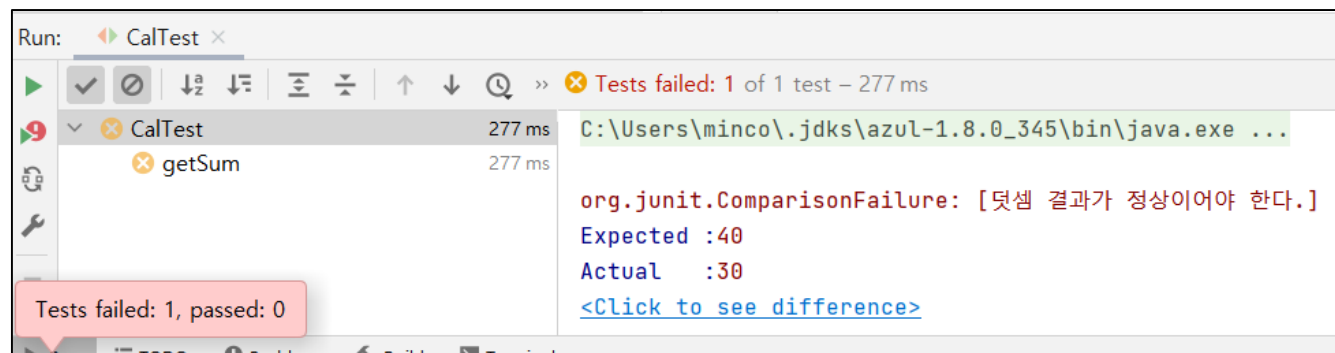

Assertion description

as(설명)

에러에 대한 설명 기록

```
@Test
public void getSum() {
    Cal c = new Cal();
    int ret = c.getSum(a: 10, b: 20);

    assertThat(ret).as(description: "덧셈 결과가 정상이어야 한다.").isEqualTo(40);
}
```



수 검증

25보다 크고

35보다 작고

31이 아니면 OK

```
Cal c = new Cal();  
int ret = c.getSum( a: 10, b: 20);  
  
assertThat(ret) AbstractIntegerAssert<capture of ?>  
    .isGreaterThan(25) capture of ?  
    .isLessThan(35)  
    .isNotEqualTo(31);
```

문자열 Assertion

다음 예제를 이해해 보자.

```
assertThat( actual: "ABCCCCDE")  
    .startsWith("AB")  
    .endsWith("DE");
```

AB로 시작해서
DE로 끝나야 한다.

```
assertThat( actual: "The Lord of the Rings").isNotNull()  
    .startsWith("The")  
    .contains("Lord")  
    .endsWith("Rings");
```

Null 이 아니어야 하고,
The로 시작하고
Lord 단어가 있어야 하고
Rings로 끝나야 한다.

배열 Assertion

배열 내 데이터가 존재하는지 검사

```
ArrayList<Integer> arr = new ArrayList<>();  
arr.add(3);  
arr.add(4);  
arr.add(5);  
  
assertThat(arr).contains(3);  
assertThat(arr).contains(5);  
assertThat(arr).doesNotContain(77);
```

[도전] AssertJ 사용하기

이름을 넣으면

거꾸로된 이름을 Return 해주는 모듈

- 예시1 : ABC → CBA
- 예시2 : MMHE → EHMM

예외검증 1

Try ~ Catch 를 이용한
예외 검증

```
@Test
public void testExceptionMessage() {

    try {
        ArrayList<Integer> arr = new ArrayList<>();
        arr.get(0);
    }
    catch (IndexOutOfBoundsException e) {
        assertThat(e.getMessage()).contains("Index: 0");
    }

}
```

예외검증 2

람다식 안에서, 예외가 발생한다면
예외 메시지에 "index:0"이 존재하는지 검사한다.

```
@Test
public void testExceptionMessage() {

    assertThatThrownBy(() -> {
        ArrayList<Integer> arr = new ArrayList<>();
        arr.get(0);
    })
        .assertInstanceOf(IndexOutOfBoundsException.class)
        .hasMessageContaining("Index: 0");
}
```

assertThatThrownBy() 사용

```
@Test
public void testExceptionMessage() {

    assertThatIndexOutOfBoundsException().isThrownBy(() -> {
        ArrayList<Integer> arr = new ArrayList<>();
        arr.get(0);
    })
        .withMessageContaining("Index: 0");
}
```

assertThatIndexOutOfBoundsException() 사용

[도전] AssertJ 사용하기 – Exception 추가

이름을 넣으면

거꾸로된 이름을 Return 해주는 모듈

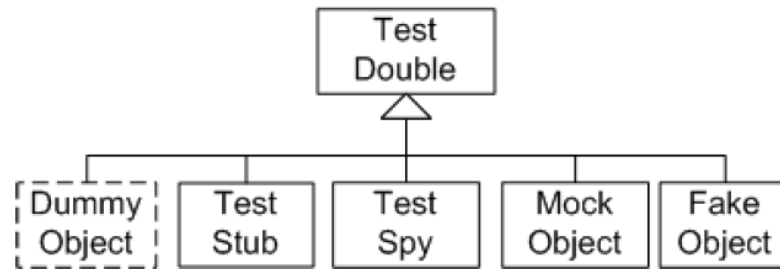
- null 입력시 : NullPointerException
- 비어있는 text 입력시 : IllegalArgumentException 발생

Test Double (구어로 Mocking이라고 한다.)

Test Double (Mocking)

테스트 더블

객체 행동 흉내내기 위함 (스턴트맨과 같은 대역)
일반적으로 다른 객체를 테스트하기 위해 만든다.



테스트 더블 쓰는 이유?

유닛 테스트하기 어려울 때는 많이 발생 한다.

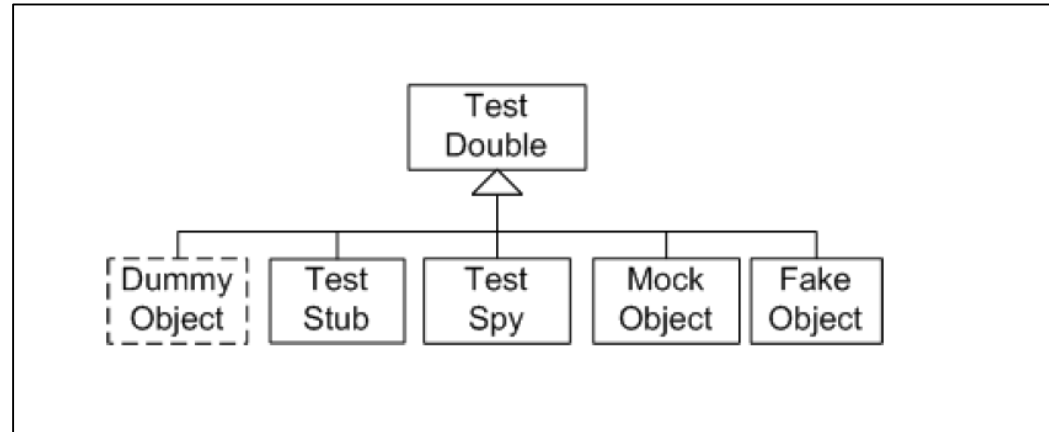
- 서비스를 중단 시키고, 유닛테스트를 해야한다면? → 대역필요
- 오픈시간 등, 제약사항 있는 외부 API를 사용해야한다면? → 대역 필요
- 고객 DB 접근 모듈을 테스트 해야한다면? → 대역 필요
- 미완성된 모듈을 포함해서 테스트해야한다면? → 대역필요

또는 유닛테스트가 너무 느리다면?

→ 대역 필요

하나씩 의미를 살펴보자.

1. Dummy
2. Stub
3. Spy
4. Mock Object
5. Fake Object



더미 (Dummy)

동작만을 위해 아무런 의미 없이 만드는 객체

- issueTo()
 - Argument에 누가 발행했는지 넣어줘야 한다.
 - 넣는 대상은, 테스트에 관심사가 아니다.
- numberOfTimesIssued()
 - 몇 번 Issue (발행)했는지 확인

```
Book javaBook = new Book("Java 101", "123456");  
  
Member dummyMember = new DummyMember();  
  
javaBook.issueTo(dummyMember);  
  
assertEquals(1, javaBook.numberOfTimesIssued());
```

스텝(Stub)

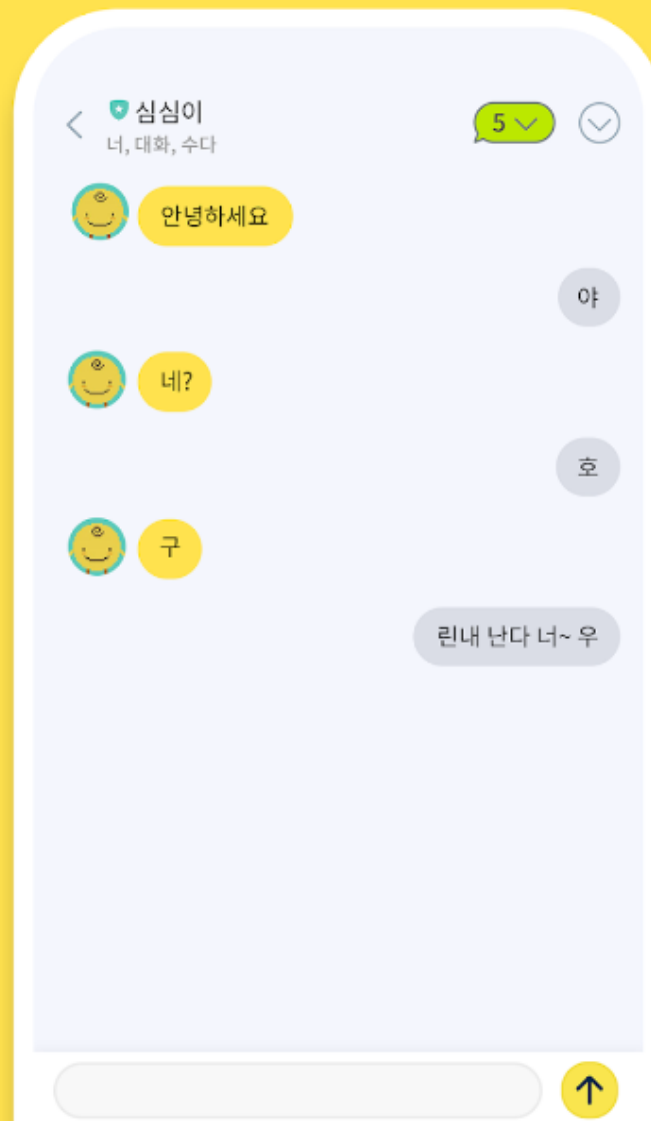
테스트로 작성된 호출에
미리 준비된 응답을
해주는 **기능을 뜻하는 용어**이다.

Stub Object

- 더미인데,
실제로 동작되는 것 처럼 만든 객체

심심할 땐 심심이!

심심이에게는 무슨 말이든 할 수 있어요.



스파이 (Spy)

실제 객체와 완전 동일하다.

스텝 기능도 있다.

그리고 Behavior(행동) 검증도 가능하다.

스텝 예시

- 배열에다가 add("안녕") 를 호출하면 false를 리턴하도록 세팅
- 배열에다가 add("뭐해") 를 호출하면 true를 리턴하도록 세팅

Behavior 검증 예시

- 총 몇 번 호출되었는지 검사
- 순차적으로 이런 메서드가 호출되었는지 검사

스파이 예시

실제와 같은 SPY 객체를 만들어두고,
몰래 역할을 부여한다.

- 실제와 똑같이 행동하도록 해.
- 그런데 만약 이렇게 요청이 들어오면 이렇게 대답하도록 해

서비스와 같은 행동이 다 끝나고,
정보를 몰래 가서 물어본다.

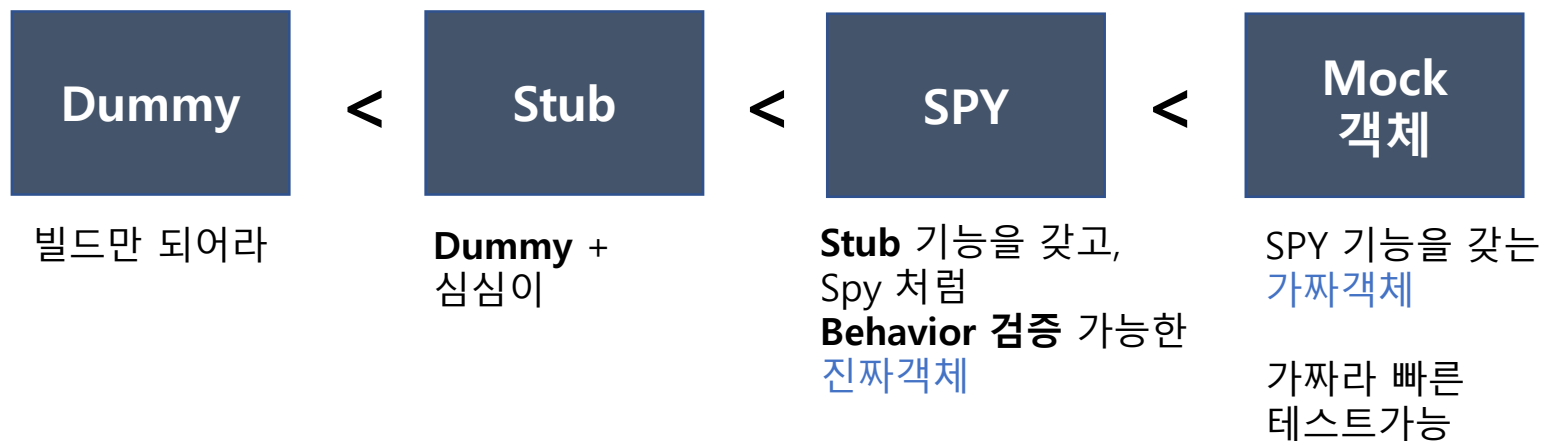
- 서비스 도중에, 이 함수 총 몇 번 호출되었어?

목 (Mock)

SPY와 같은 기능을 가진다. (Stub / Behavior 검증 가능)
그런데 SPY와 달리, 실제 객체가 아니라 **가짜 객체**이다.

정리

네 가지 Test Double 정리



Q&A) 전부 SPY 만 쓰면 되지않나?

Mock 은 가짜고,
SPY 는 진짜에 감시 기능을 심는 것이면,
진짜로 테스트하는게 무조건 좋은게 아닌가?

1. API 외부 접속 등.. 상황이 어려울때 Mocking을 쓰는 것인데,
진짜 객체가 필요하면 아예 Mocking 을 안했을 것이다.
2. 해당 객체 검증이 중요하지 않다면,
가짜로 만드는 것이 테스트가 더 빨라질 것이다.

Fake object

실제로 동작하는 객체인데,

테스트용으로 만든 가짜 객체

- 가짜 데이터 들을 모두 만들어 두어야한다.

예시

- DB를 제어하는 Class를 유닛테스트를 하고자 한다.
- MySQL 의 고객 DB Data 를 그대로 사용할 수 없으니까,
테스트를 위한, 테스트용 DB 구축 및 DB 객체를 만들어두고,
이것으로 UnitTest를 한다.

**실제로 잘 동작되는,
그리고 테스트를 위해서 만든 객체를 Fake Object 라고 한다.**

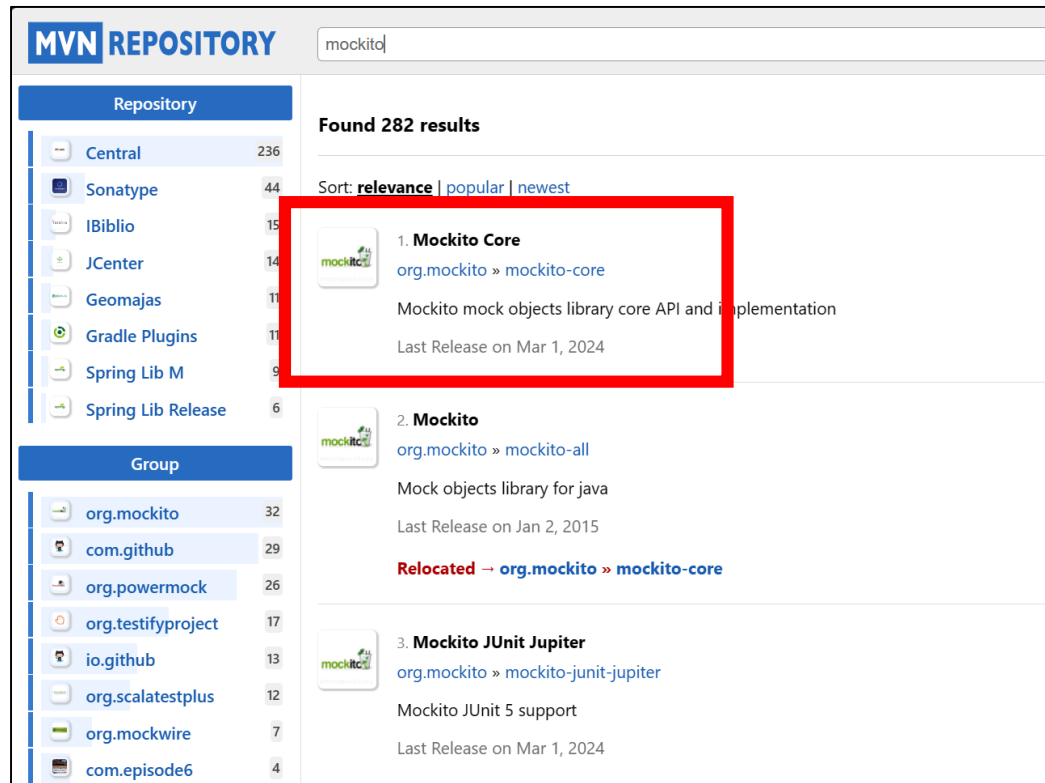
Maven을 이용한 설치 및 기본 테스트

Mockito 준비

Mockito 설치 방법

✓설치 방법

- MVN Repository 접속 후 Mockito Core로 Dependency 를 이용한 설치
- Java 1.8 인 경우는 Mockito 4.8 버전을 써야한다.



Mockito 시작

@ExtendWith 어노테이션

- Mockito Start

@Mock

- mock 객체 생성

Mockito.verify()

- 오른쪽 예시는 **static import** 등록한 것
- add 되었는지 확인 가능

빨간색 단어는 Alt + Enter로 해결하자.

```
@ExtendWith(MockitoExtension.class) //JUnit 4에서는 @RunWith(MockitoJUnitRunner.class)이었음
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("one");
        mockList.add("one");
        mockList.add("one");

        // 적어도 세번 이상 add("one")을 수행했는지 검사
        verify(mockList, atLeast( minNumberOfInvocations: 3)).add("one");
    }
}
```

먼저 Mockito.verify 라고 기입 후,
Mockito를 블록 잡고
Alt + Enter을 눌러 static import로 등록하자.

[참고] 완성된 초기 코드 (import 포함)

✓수동 입력없이
Alt + Enter로 모두 등록하였음

✓만약 에러가 발생한다면
오른쪽과 다른 import가 있는지
확인하자.

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.ArrayList;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class) //JUnit 4 에서는 @RunWith(MockitoJUnitRunner.class) 이었음
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("one");
        mockList.add("one");
        mockList.add("one");

        //적어도 세번 이상 add("one")을 수행했는지 검사
        verify(mockList, atLeast(minNumberOfInvocations: 3)).add("one");
    }
}
```


mock 객체 생성 방법

둘 다 많이 사용 됨!

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Test
    void getSum() {
        ArrayList<String> mockList = mock(ArrayList.class);
        mockList.add("HI");

        verify(mockList).add("HI");
    }
}
```

mock 객체 생성 방법
(지역 변수로 객체 생성)

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("HI");

        verify(mockList).add("HI");
    }
}
```

필드로 생성 방법
(어노테이션 사용)

Mocking시, 가장 많이 사용되는 두 가지 기능
Stubbing & Verify

출력결과는 null

mock 객체는 가짜로 동작

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("one");

        System.out.println(mockList.get(0));
    }
}
```

Stubbing

소스코드 해석

when(mockList).get(0); 했을 때
doReturn("OH MY GOD")
을 수행한다.

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("one");

        doReturn(toBeReturned: "OH MY GOD").when(mockList).get(0);
    }
}
```

Stub을 걸었지만, 사용하지않았기에
UnnecessaryStubbingException 이 발생한다.

lenient() 추가하기

✓사용되지 않을 수도 있다는
lenient(허술한)
함수를 하나 추가해준다.

✓정상동작된다.

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("one");

        lenient().doReturn("OH MY GOD").when(mockList).get(0);
    }
}
```

수행결과

- ✓Stubbing을 하였고, 출력하면 Stub한 대로 출력된다.
 - 실제 사용했으니, lenient() 는 제거해도 잘 동작된다.

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("one");

        lenient().doReturn("OH MY GOD").when(mockList).get(0);

        System.out.println(mockList.get(0));
    }
}
```

OH MY GOD

when

doReturn() : 내부적으로 get(0)를 호출 안 함

thenReturn() : 내부적으로 get(0)를 호출 함

- 차이만 알고 편한 것으로 사용하자.

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    ArrayList<String> mockList;

    @Test
    void getSum() {
        mockList.add("one");

        lenient().doReturn(o: "OH MY GOD").when(mockList).get(0);

        System.out.println(mockList.get(0));
    }
}
```

doReturn() 이후 when()

```
@Test
void getSum() {
    mockList.add("one");

    //doReturn, 내부적으로 get(0)를 호출 안함
    lenient().doReturn(o: "OH MY GOD").when(mockList).get(0);

    //thenReturn, 내부적으로 get(2)를 호출 함
    lenient().when(mockList.get(2)).thenReturn(t: "FLOWER");

    System.out.println(mockList.get(0));
    System.out.println(mockList.get(2));
}
}
```

when() 이후 thenReturn()

OH MY GOD
FLOWER

[도전] 처음부터 구현해보기

ArrayList<Integer> mock 객체 만들고

size() 를 물어보면 10000을 리턴하는 객체 만들기

그리고 assertEquals로 size() 시 10000 이 맞는지 검사하기

mock 객체 특징

실제로 해당 객체를 사용하지 않는다.

```
@Test
public void test32() {
    mockList.add("one");
    mockList.add("two");
    mockList.add("one");

    System.out.println(mockList.get(0));
    System.out.println(mockList.size());
}
```

null
0

times 옵션

총 몇 번 호출했는지 검증

```
@Test
public void test32() {
    mockList.add("one");
    mockList.add("two");
    mockList.add("one");
    mockList.add("one");

    int size1 = mockList.size();
    int size2 = mockList.size();
    int size3 = mockList.size();

    verify(mockList, times(wantedNumberOfInvocations: 3)).size();
}
```

[도전] atMost 옵션

size() 호출 횟수가

atLeast 2번 이상이고

atMost 5번 이하일때 PASS 이고

그렇지 않으면 FAIL 이 발생하도록 만들기

- 힌트 : times 대신 atLeast / atMost 사용하면 된다.

verify 사용하기

anyString() : 아무 문자열을 나타냄.

never() : 사용한적 없어야 한다.

```
@Test
public void test32() {
    mockList.add("one");
    mockList.add("two");
    mockList.add("one");
    mockList.add("one");

    //2 ~ 4 회 아무 String 이나 add가 되었어야한다.
    verify(mockList, atLeast( minNumberOfInvocations: 2)).add(anyString());
    verify(mockList, atMost( maxNumberOfInvocations: 4)).add(anyString());

    //clear() 를 한적이 없어야한다.
    verify(mockList, never()).clear();
}
```

[참고] 순서대로 호출되었는지 확인

InOrder 객체 사용

```
@Mock
ArrayList<String> mockList;

@Test
public void test32() {
    mockList.add("one");
    mockList.add("two");
    mockList.clear();
    mockList.add("three");

    InOrder order = inOrder(mockList);
    order.verify(mockList).add("one");
    order.verify(mockList).add("two");
    order.verify(mockList).clear();
    order.verify(mockList).add("three");
}
```

[정리] Stubbing vs Behavior Verification

Stubbing

- A 값을 Parameter로 넣으면, B 값이 return 되도록 세팅

Behavior Verification

- Test 종료 후, 특정 메서드가 동작 되었는지 검증 가능
- 호출 횟수 / 순서대로 호출 여부 등 확인 가능

Stubbing 응용

doReturn / thenReturn()

편한 것으로 사용

doReturn

- 내부적으로 add를 호출 안함

thenReturn

- 내부적으로 add 호출

```
@Mock
ArrayList<String> mockList;

@Test
public void test32() {
    doReturn( toBeReturned: false).when(mockList).add("one");
    System.out.println( mockList.add("one") );

    when(mockList.add("one")).thenReturn(true);
    System.out.println( mockList.add("two") );
}
```

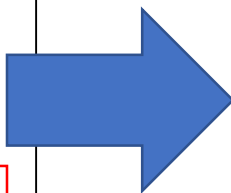

doThrow / thenThrow()

예외가 발생하도록 한다.

```
@Test
public void test32() {
    doReturn( toBeReturned: false).when(mockList).add("one");
    System.out.println( mockList.add("one") );

    when(mockList.add("one")).thenReturn(true);
    System.out.println( mockList.add("two") );

    doThrow(IllegalStateException.class).when(mockList).add("three");
    System.out.println( mockList.add("three") );
}
```



```
@Test(expected = IllegalStateException.class)
public void test32() {
    doReturn( toBeReturned: false).when(mockList).add("one");
    System.out.println( mockList.add("one") );

    when(mockList.add("one")).thenReturn(true);
    System.out.println( mockList.add("two") );

    doThrow(IllegalStateException.class).when(mockList).add("three");
    System.out.println( mockList.add("three") );
}
```

예외가 잘 발생되는지 확인해본다.

예외 발생시 Pass가 되도록 세팅

순차적 검사

순차적으로 add를 해보자.

- 한번 더 add를 했을 때 어떻게 되는지 확인하자.

```
@Test
public void test32() {
    when(mockList.add("date me?"))
        .thenReturn(false)
        .thenReturn(false)
        .thenReturn(false)
        .thenReturn(false)
        .thenReturn(false)
        .thenReturn(true)
        .thenThrow(IllegalStateException.class);

    mockList.add("date me?");
    mockList.add("date me?");
    mockList.add("date me?");
    mockList.add("date me?");
    mockList.add("date me?");
    mockList.add("date me?");
}
```

가짜가 아닌, SPY 역할을 하는 진짜 객체

SPY 이해

[복습] mock 객체 특징

실제로 해당 객체를 사용하지 않는다.

```
@Test
public void test32() {
    mockList.add("one");
    mockList.add("two");
    mockList.add("one");

    System.out.println(mockList.get(0));
    System.out.println(mockList.size());
}
```

null
0

Spy

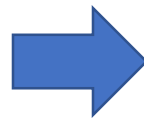
Mock 객체와 달리
실제 객체로 동작된다.

```
@Mock
ArrayList<String> mockList = new ArrayList<>();

@Test
public void test32() {
    mockList.add("one");
    mockList.add("two");
    mockList.add("one");

    System.out.println(mockList.get(0));
    System.out.println(mockList.size());
}
```

null
0



```
@Spy
ArrayList<String> mockList = new ArrayList<>();

@Test
public void test32() {
    mockList.add("one");
    mockList.add("two");
    mockList.add("one");

    System.out.println(mockList.get(0));
    System.out.println(mockList.size());
}
```

one
3

[도전] 직접 작성해보기 (Mock vs Spy 비교)

Cal class 만들기

- getSum(int a, int b) : sum을 리턴한다.
- getGopThree(int a, int b) : GetSum을 3회 호출하고, 그 곱을 리턴

테스트 케이스 내용

1. Cal 목객체 생성하기
2. GetSum() 수행
3. GetGopThree() 수행
4. GetSum 이 4회 호출되었다면 Pass

Spy vs Mock

- Spy 는 4회 호출이 된다.
- Mock 은 1회 호출된다.

SPY를 쓰는 이유

Mock 과 달리, 진짜로 동작하는 객체에 Stubbing / Verify 가 필요할 때 사용된다.

- Mock 과 Spy 모두 Stubbing / Verify 가능하다.
- 하지만, 객체가 실제 동작될 필요 없다면, Spy 보다 가볍고 빠른 Mock을 사용하면 된다.

[도전] Spy로 똑같이 Stubbing 하기

1. Print 객체를 하나 만든다.
2. Spy 객체로 만든다.
3. getX를 호출하면 9999 가 나오도록 Stubbing 한다.
4. AssertEquals 로 검증해본다.

```
class Print {  
    private int x = 10;  
  
    public int getX() {  
        return x;  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
}
```


목 주입하기

Mock Injection

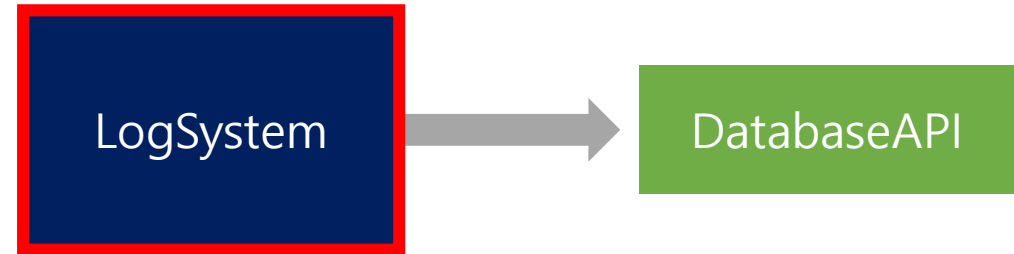
다음과 같이 구현하자.

✓테스트 하고자 하는 모듈

- LogSystem

✓LogSystem의 의존성 모듈

- DatabaseAPI



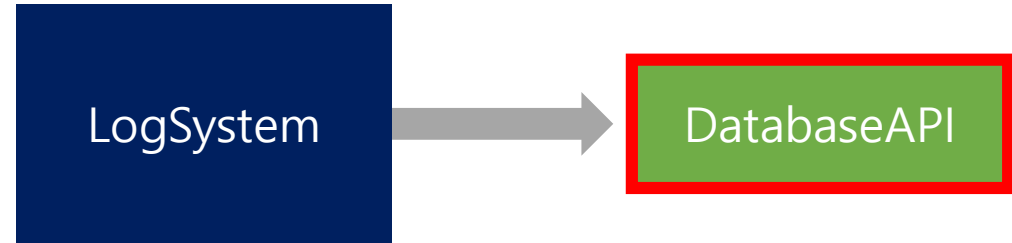
```
class LogSystem {  
    private DatabaseAPI DB;  
  
    public String getLogMessage(String content) {  
        String msg = "";  
        msg += "[" + DB.getDBName() + "]" + content;  
        return msg;  
    }  
}
```

```
class DatabaseAPI {  
    private String name = "MySon_DB";  
  
    public String getDBName() {  
        return name;  
    }  
}
```

테스트 할 때, 문제점

✓테스트 하고자 하는 모듈

- LogSystem



✓DatabaseAPI 동작

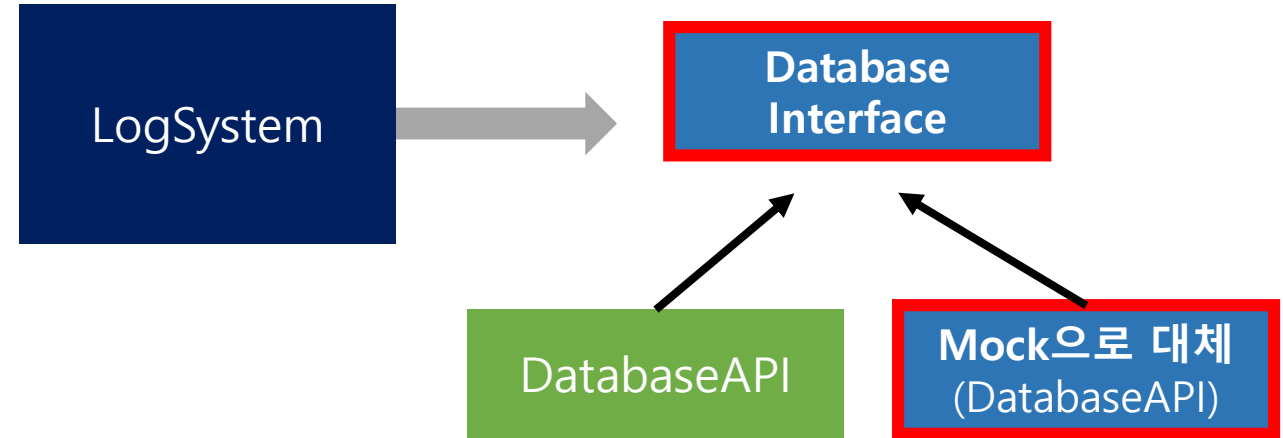
- Unit Test를 할 때 마다, Database의 **getDBName API**를 수행하고, 서버 트래픽을 발생시킨다고 가정한다.
- 이로 인해, 테스트 할 때마다 서버 성능이 낮아진다.

```
class DatabaseAPI {  
    private String name = "MySon_DB";  
  
    public String getDBName() {  
        return name;  
    }  
}
```

목표 : Mock으로 대체

✓테스트 하고자 하는 모듈

- LogSystem



✓Mocking을 하면 트래픽 문제 없이
테스트 하고자 하는 모듈을 테스트가 할 수 있다.

```
class LogSystem {  
    private DatabaseAPI DB;  
  
    public String getLogMessage(String content) {  
        String msg = "";  
        msg += "[" + DB.getDBName() + "]" + content;  
        return msg;  
    }  
}
```

이 DB 객체를 Mock객체로 변경필요

Mock으로 대체를 위한 준비작업

- ✓ Database 객체를
내부에서 생성하는 것이 아닌,
밖에서 주입해주는, DI 로 리팩토링

```
interface API {  
    public String getDBName();  
}  
  
class DatabaseAPI implements API {  
    private String name = "MySon_DB";  
  
    public String getDBName() {  
        return name;  
    }  
}
```

```
class LogSystem {  
    private API DB;  
  
    public LogSystem(API DB) {  
        this.DB = DB;  
    }  
  
    public String getLogMessage(String content) {  
        String msg = "";  
        msg += "[" + DB.getDBName() + " ] " + content;  
        return msg;  
    }  
}
```

유닛테스트가 가능하도록 리팩토링 하는 것이다.

Mock 주입하기

✓Mock 객체를 준비하여, DI 해준다.

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    API apiMock;

    @Test
    void name() {
        doReturn("TEST DOUBLE").when(apiMock).getDBName();

        LogSystem app = new LogSystem(apiMock);
        System.out.println(app.getLogMessage("HI"));
    }
}
```

✓ CalTest	246 ms
✓ name()	246 ms

✓ Tests passed: 1 of 1 test –

C:\Users\User\.jdk\azu

[TEST DOUBLE] HI

[참고] @InjectMocks

Mock을 간단하게 주입하도록 만들어 준다.

- 실행결과는 이전과 동일해야 함

```
@ExtendWith(MockitoExtension.class)
class CalTest {

    @Mock
    API apiMock;

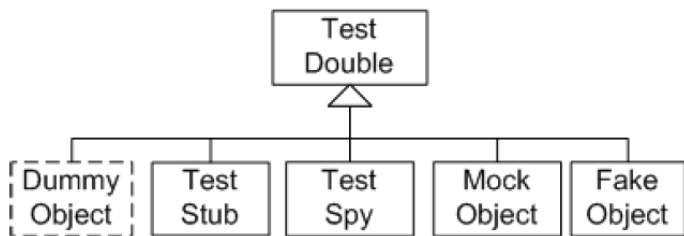
    @InjectMocks
    LogSystem app;

    @Test
    void name() {
        doReturn("TEST DOUBLE").when(apiMock).getDBName();
        System.out.println(app.getLogMessage("HI"));
    }
}
```

[중요] Mock Library를 써야 하는 이유

Mock Library를 쓰는 이유

- 직접 만들면, 개발자 마다 구현 방법이 모두 다르다. (유지보수의 어려움)
- 가장 유명한 Library를 사용하면, 같은 방법으로 구현한다.



Dummy > Stub > Spy > Mock || Fake

이론적 Test Double 구성

